МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность
Образовательная программа: «Технологии защиты информации»

Дисциплина:
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Разработка программных средств разграничения доступа к локальным файловым объектам и защиты веб-контента от несанкционированного копирования»

Выполнил студент:

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Проверил:

Ищенко Алексей Петрович / _____

ФИО *Подпись*

_____

Отметка о выполнении

_____

Дата

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 АРХИТЕКТУРА И МЕХАНИЗМЫ ЛОКАЛЬНОЙ ЗАЩИТЫ ФАЙЛОВ **4**

1.1 Принципы разграничения доступа на уровне ядра Linux 4

1.2 Формат хранения конфигурации и криптографическая защита 4

1.3 Алгоритм работы утилиты контроля 4

2 АРХИТЕКТУРА И МЕХАНИЗМЫ ЗАЩИТЫ ВЕБ-КОНТЕНТА **5**

2.1 Векторы утечки информации в браузерной среде 5

2.2 Механизмы подавления копирования и сохранения 5

3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ **6**

3.1 Эксплуатация локальной программы защиты (Задание 1а) 6

3.2 Эксплуатация веб-скрипта защиты (Задание 1б) 6

ЗАКЛЮЧЕНИЕ **8**

ПРИЛОЖЕНИЕ А Листинги исходного кода программного комплекса **9**

ВВЕДЕНИЕ

Цель работы: приобретение практических навыков реализации программных механизмов защиты информации на уровне файловой системы операционной системы Linux и в среде клиентских веб-приложений; разработка локального сервиса контроля операций с файловыми объектами на базе шаблонов с аутентификацией по паролю, а также реализация внедряемого сценария защиты веб-документов от несанкционированного копирования, выделения и локального сохранения при гарантированном сохранении функции вывода на печать.

Постановка задачи:

1. **Задание 1(а) (Локальная программа)**:
    - Разработать программу, запрещающую в текущем рабочем каталоге создание, копирование, удаление или переименование файлов с заданными именами или масками.
    - Список имен и шаблонов масок хранить в текстовом конфигурационном файле `template.tbl`.
    - Реализовать многоуровневую защиту файла `template.tbl` от удаления, несанкционированного чтения и модификации непривилегированными пользователями.
    - Предусмотреть отключение защиты только после ввода пароля, хеш-сумма которого сохраняется в первой строке файла `template.tbl`.
    - Реализовать режимы включения и выключения защитного механизма через командный интерфейс.
2. **Задание 1(б) (Веб-скрипт)**:
    - Разработать клиентский сценарий (`JavaScript`), встраиваемый в HTML-документ и блокирующий копирование содержимого в буфер обмена, выделение текста и захват данных через скриншоты.
    - Заблокировать сохранение страниц на диск (комбинация клавиш `Ctrl+S` и эквиваленты) для текущей страницы и всех связанных гиперссылками страниц цепочки.
    - Обеспечить беспрепятственную доступность вывода защищаемых страниц на печать.
    - Реализовать отключение защиты по мастер-паролю, хранящемуся в теле скрипта в зашифрованном/хешированном виде.

1 АРХИТЕКТУРА И МЕХАНИЗМЫ ЛОКАЛЬНОЙ ЗАЩИТЫ ФАЙЛОВ

1.1 Принципы разграничения доступа на уровне ядра Linux

Для пресечения операций создания, копирования и модификации файлов в ОС Linux используется комбинированный подход:

1. **Атрибуты файловых систем ext4/xfs (бит неизменяемости)**: утилита `chattr +i` активирует флаг `FS_IMMUTABLE_FL` в дескрипторе (*inode*) файла. При активном флаге ядро Linux блокирует любые попытки изменения, удаления, переименования или создания ссылок даже для суперпользователя (`root`), пока атрибут не будет снят.
2. **Подсистема событий ядра inotify**: отслеживает операции в каталоге в реальном времени. При попытке записи нового файла ядро формирует синхронные события `IN_CREATE` и `IN_MOVED_TO`. Демон защиты сверяет имя файла с базой масок и немедленно удаляет запрещенный объект.

1.2 Формат хранения конфигурации и криптографическая защита

Конфигурационный файл `template.tbl` размещается в рабочем каталоге. Спецификация структуры файла представлена в таблице 1.

Таблица 1 – Структура конфигурационного файла `template.tbl`

№ строки	Поле	Назначение и формат представления
1	Хеш пароля	Запись вида `<salt_hex>$<hash_hex>`, сформированная алгоритмом PBKDF2-HMAC-SHA256 (100 000 итераций)
2…*N*	Маски файлов	Шаблоны имен файлов в синтаксисе Unix shell (`*.secret`, `private.txt`)

Защита конфигурационного файла обеспечивается двумя взаимодополняющими барьерами:

- Права доступа `0600` (`chmod 600`): чтение и запись разрешены исключительно владельцу `root`;
- Атрибут неизменяемости `+i` (`chattr +i`): блокирует удаление и изменение файла драйвером файловой системы при активной защите.

1.3 Алгоритм работы утилиты контроля

Программа `lab1.py` реализует четыре основных режима:

1. `init`: генерация случайной соли (16 байт), вычисление хеша PBKDF2, сохранение масок и назначение прав доступа `0600`;
2. `enable`: проверка прав администратора, установка флага `+i` на целевые файлы и `template.tbl`, запуск фонового процесса мониторинга через `os.fork()`;
3. `status`: вывод текущего состояния системы (активна/неактивна) и PID демона;
4. `disable`: запрос пароля, его валидация с защитой от атак по времени через `compare_digest`, остановка демона по сигналу `SIGTERM` и снятие атрибутов `-i`.

2 АРХИТЕКТУРА И МЕХАНИЗМЫ ЗАЩИТЫ ВЕБ-КОНТЕНТА

2.1 Векторы утечки информации в браузерной среде

При загрузке веб-страницы несанкционированное копирование информации может осуществляться следующими способами:

- Выделение текста мышью с последующим копированием в буфер (`Ctrl+C`);
- Вызов контекстного меню браузера для сохранения элементов и графики;
- Локальное сохранение страницы и сопутствующих ресурсов (`Ctrl+S`);
- Анализ разметки через инструменты разработчика (`F12`, `Ctrl+U`);
- Снятие снимка экрана с использованием системной клавиши `PrintScreen`.

2.2 Механизмы подавления копирования и сохранения

Внедряемый модуль `lab1.js` реализует эшелонированный комплекс ограничений:

1. **CSS-ограничение**: директива `user-select: none !important` полностью исключает возможность выделения текста.
2. **Перехват событий**: отмена стандартного поведения (`preventDefault()`) для событий `contextmenu`, `selectstart`, `dragstart`, `copy` и `cut`. Буфер обмена принудительно очищается через `clipboardData.setData('text/plain', '')`.
3. **Блокировка клавиатуры**: перехват событий `keydown` на этапе погружения (*capture phase*), отменяющий сочетания `Ctrl+S`, `Ctrl+U`, `Ctrl+C`, `Ctrl+X`, `F12`, а также очищающий системный буфер при нажатии `PrintScreen`.
4. **Обеспечение легитимной печати**: директива `@media print` восстанавливает стили страницы, а сочетание `Ctrl+P` исключено из списка блокировок в строгом соответствии с заданием.
5. **Отключение защиты по паролю**: в коде скрипта зафиксирован SHA-256 хеш пароля. При нажатии сочетания `Alt+D` вызывается окно ввода пароля, хеш сверяется через Web Crypto API, и при успехе ограничения отключаются на время текущей сессии (`sessionStorage`).

3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Эксплуатация локальной программы защиты (Задание 1а)

Все операции управления защитой выполняются с правами суперпользователя.

1. Инициализация параметров:

```
sudo python3 lab1.py init
```

В интерактивном режиме задается мастер-пароль и маски целевых файлов (`*.secret`, `private.txt`). Создается конфигурационный файл `template.tbl` с правами доступа `600`.

2. Включение защиты:

```
sudo python3 lab1.py enable
```

Программа накладывает флаг `+i` на существующие файлы, блокирует конфигурационный файл и переводит процесс мониторинга в фоновый режим.

3. Проверка эффективности защиты: Попытка создания защищаемого файла:

```
echo "confidential data" > text.secret
# Файл мгновенно удаляется демоном inotify
```

Попытка удаления конфигурационного файла:

```
rm -f template.tbl
# rm: cannot remove 'template.tbl': Operation not permitted
```

Попытки копирования и перемещения защищенных файлов:

```
cp text.txt text2.txt
# cp: cannot create regular file 'text2.txt': Operation not permitted
mv text.txt text2.txt
# mv: replace 'text2.txt', overriding mode 0644 (rw-r--r--)?
```

4. Отключение защиты:

```
sudo python3 lab1.py disable
```

После успешной аутентификации фоновый процесс завершается, атрибуты `-i` снимаются, и доступ восстанавливается в полном объеме.

3.2 Эксплуатация веб-скрипта защиты (Задание 1б)

Для демонстрации подготовлен комплекс веб-страниц: `index.html`, `page2.html`, `page3.html`, к каждой из которых подключен скрипт `lab1.js`.

- **Блокировка действий**: выделение текста заблокировано, клик правой кнопкой мыши не вызывает контекстное меню. Нажатие `Ctrl+S` прерывается диалоговым окном с уведомлением о блокировке.
- **Наследование защиты**: при переходе по гиперссылкам на страницы `page2.html` и `page3.html` защитные свойства сохраняются.
- **Доступность вывода на печать**: при нажатии сочетания клавиш `Ctrl+P` открывается стандартное системное окно печати с корректным отображением контента (Рисунок 1).

- **Деактивация защиты**: по нажатию `Alt+D` запрашивается мастер-пароль. После ввода пароля (`admin`) блокировки отключаются до конца работы с браузером (Рисунок 2).

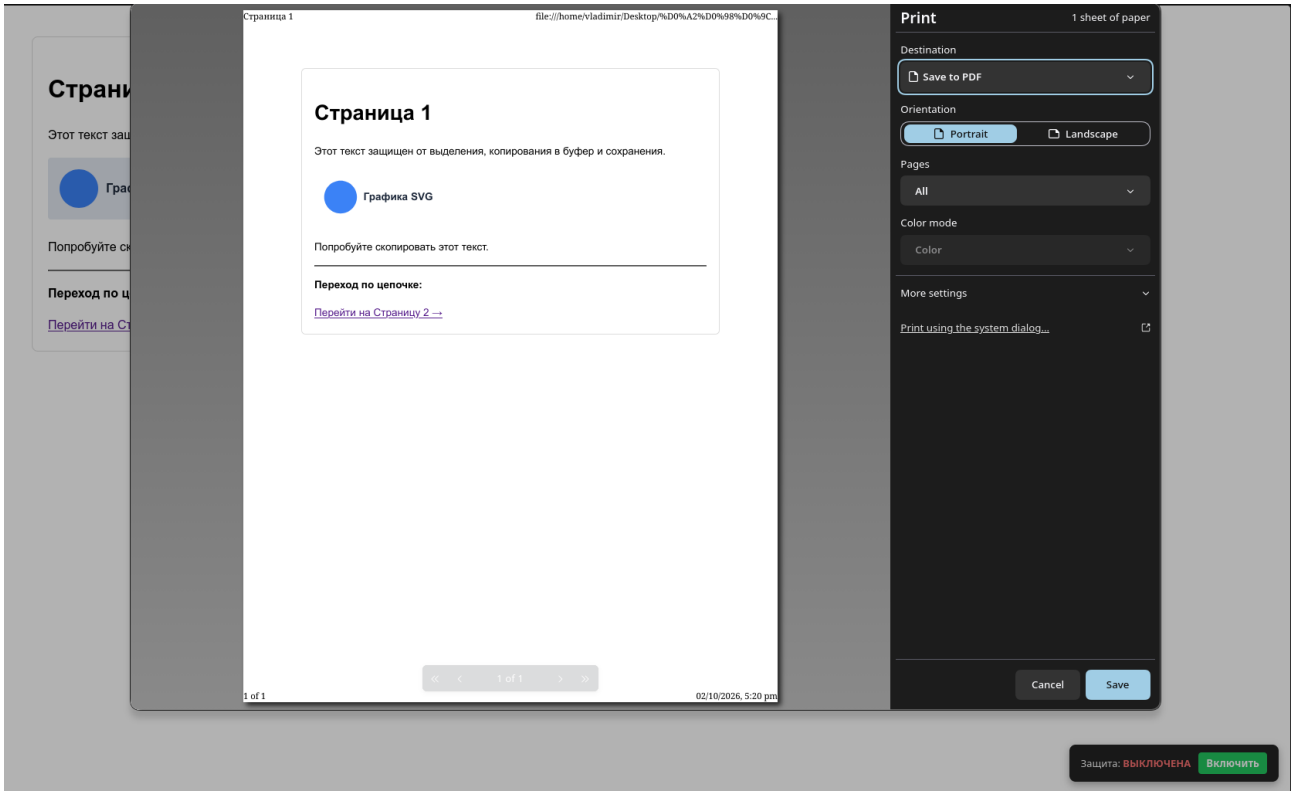


Рисунок 1 – Демонстрация вывода на печать

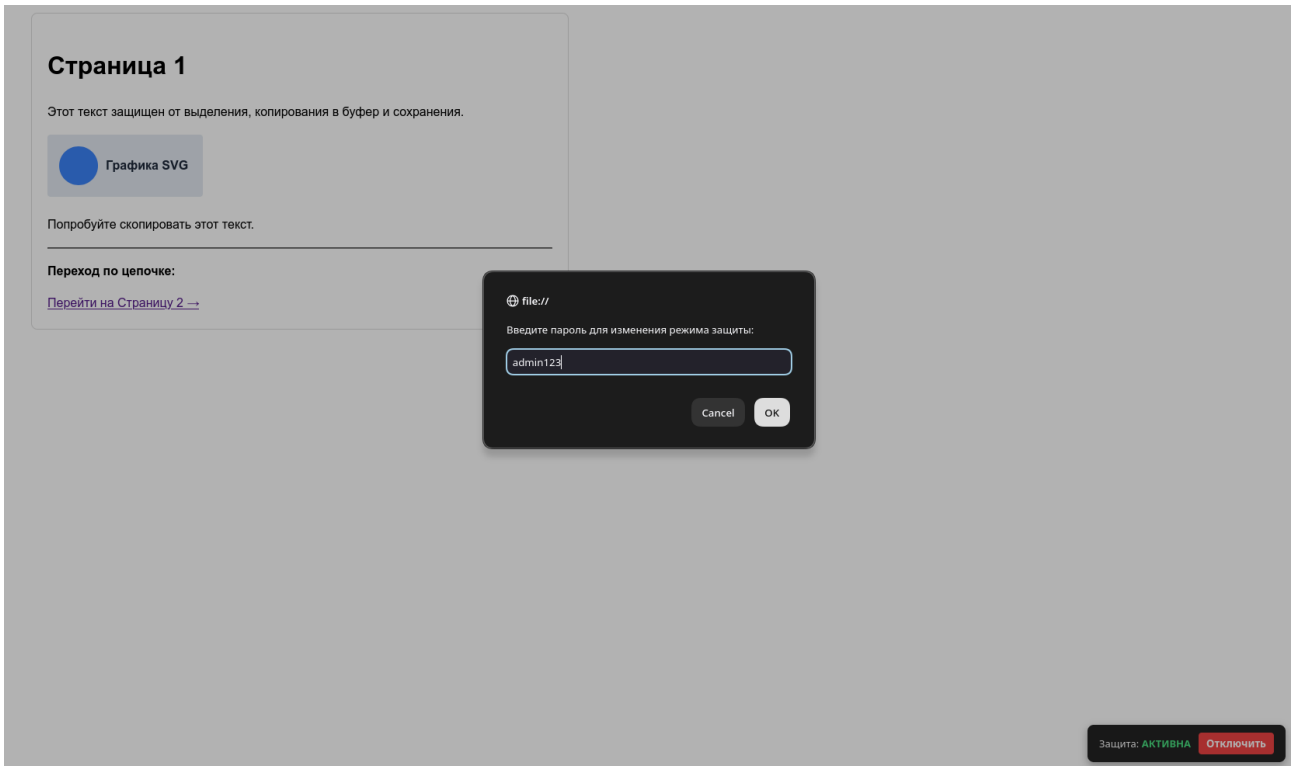


Рисунок 2 – Демонстрация работы веб-скрипта защиты контента

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были исследованы и реализованы прикладные методы защиты цифровых ресурсов в операционных системах семейства Linux и клиентских веб-документах.

Основные результаты работы:

1. Разработана системная утилита `lab1.py`, сочетающая атрибуты неизменяемости файловой системы (`chattr +i`) и реактивный перехват системных событий ядра Linux посредством интерфейса `inotify`.
2. Реализована защищенная конфигурация `template.tbl` с правами доступа `0600` и хранением пароля в виде криптографического хеша PBKDF2-HMAC-SHA256 с индивидуальной солью.
3. Создан внедряемый веб-модуль `lab1.js`, нейтрализующий распространенные векторы утечки контента (выделение текста, контекстное меню, копирование в буфер, сохранение DOM-структуры, снимки экрана).
4. Обеспечено наследование защитных функций по всей цепочке связанных гиперссылками страниц с гарантированным сохранением возможности вывода документов на печать.
5. Реализован механизм аутентифицированного отключения веб-защиты на основе вычисления дайджеста SHA-256.

ПРИЛОЖЕНИЕ А

Листинги исходного кода программного комплекса

Листинг 1. Скрипт локальной защиты каталога (lab1.py)

```
#!/usr/bin/env python3
import os
import sys
import fnmatch
import hashlib
import secrets
import subprocess
import getpass
import signal
import ctypes
import struct
import argparse
from pathlib import Path

DIR_PATH = Path(__file__).resolve().parent
TBL_FILE = DIR_PATH / "template.tbl"
PID_FILE = DIR_PATH / ".lab1.pid"

IN_CREATE = 0x00000100
IN_MOVED_TO = 0x00000080
EVENT_FMT = "iIII"
EVENT_SIZE = struct.calcsize(EVENT_FMT)

libc = ctypes.CDLL("libc.so.6", use_errno=True)


def check_root():
    if os.geteuid() != 0:
        print("[!] Ошибка: Для работы с атрибутами безопасности и chattr
    требуются права root.")
        print("    Запустите скрипт через sudo: sudo python3 lab1.py команда<>")
        sys.exit(1)


def set_immutable(path: Path, enable: bool):
    if path.exists():
        flag = "+i" if enable else "-i"
        subprocess.run(["chattr", flag, str(path)], check=False, stderr=
    subprocess.DEVNULL)


def hash_password(password: str, salt: bytes = None) -> str:
    if salt is None:
```

```
        salt = secrets.token_bytes(16)
    pwd_hash = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000)
    return f"{salt.hex()}${pwd_hash.hex()}"


def verify_password(password: str, stored_record: str) -> bool:
    try:
        salt_hex, hash_hex = stored_record.strip().split("$")
        salt = bytes.fromhex(salt_hex)
        test_hash = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100
    _000)
        return secrets.compare_digest(test_hash.hex(), hash_hex)
    except Exception:
        return False


def load_templates():
    if not TBL_FILE.exists():
        print(f"[!] Файл конфигурации {TBL_FILE.name} не найден. Сначала
    выполните init.")
        sys.exit(1)

    with open(TBL_FILE, "r", encoding="utf-8") as f:
        lines = [line.strip() for line in f if line.strip()]

    if not lines:
        print(f"[!] Файл {TBL_FILE.name} пуст.")
        sys.exit(1)

    pwd_hash = lines[0]
    masks = lines[1:]
    return pwd_hash, masks


def matches_any_mask(filename: str, masks: list) -> bool:
    return any(fnmatch.fnmatch(filename, mask) for mask in masks)


def init_program():
    check_root()

    if TBL_FILE.exists():
        set_immutable(TBL_FILE, False)

    pwd1 = getpass.getpass("Задайте мастерпароль- для отключения защиты: ")
    pwd2 = getpass.getpass("Подтвердите пароль: ")
    if pwd1 != pwd2 or not pwd1:
        print("[!] Пароли не совпадают или пустые.")
        sys.exit(1)
```

```
    pwd_record = hash_password(pwd1)

    print("\Введитеn маски или имена запрещенных файлов по( одной на строке).")
    print("Примеры: *.secret, private.txt, report_*.doc")
    print("Для завершения ввода нажмите Enter на пустой строке:")

    masks = []
    while True:
        try:
            line = input("> ").strip()
            if not line:
                break
            masks.append(line)
        except EOFError:
            break

    with open(TBL_FILE, "w", encoding="utf-8") as f:
        f.write(pwd_record + "\n")
        for mask in masks:
            f.write(mask + "\n")

    os.chmod(TBL_FILE, 0o600)
    print(f"[+] Файл {TBL_FILE.name} успешно создан и защищен правами 600.")


def daemon_monitor():
    _, masks = load_templates()

    fd = libc.inotify_init()
    if fd < 0:
        sys.exit(1)

    watch_flags = IN_CREATE | IN_MOVED_TO
    wd = libc.inotify_add_watch(fd, str(DIR_PATH).encode(), watch_flags)
    if wd < 0:
        sys.exit(1)

    while True:
        try:
            data = os.read(fd, 4096)
            if not data:
                continue

            offset = 0
            while offset < len(data):
                wd, mask, cookie, name_len = struct.unpack_from(EVENT_FMT, data,
    offset)
```

```
                name_bytes = data[offset + EVENT_SIZE : offset + EVENT_SIZE +
    name_len]
                filename = name_bytes.split(b"\x00", 1)[0].decode("utf-8",
    errors="ignore")
                offset += EVENT_SIZE + name_len

                if not filename:
                    continue

                if filename == PID_FILE.name:
                    continue

                if matches_any_mask(filename, masks):
                    target = DIR_PATH / filename
                    try:
                        if target.is_file() or target.is_symlink():
                            target.unlink()
                        elif target.is_dir():
                            target.rmdir()
                        print(f"[BLOCKED] Заблокировано созданиекопирование/: {
    filename}", flush=True)
                    except Exception as e:
                        print(f"[ERROR] {filename}: {e!r}", flush=True)
            except KeyboardInterrupt:
                break


def enable_protection():
    check_root()

    if PID_FILE.exists():
        print("[!] Защита уже активна.")
        return

    _, masks = load_templates()

    count = 0
    for item in DIR_PATH.iterdir():
        if item.is_file() and matches_any_mask(item.name, masks):
            set_immutable(item, True)
            count += 1

    os.chmod(TBL_FILE, 0o600)
    set_immutable(TBL_FILE, True)

    pid = os.fork()
    if pid == 0:
        os.setsid()
        sys.stdin = open(os.devnull, "r")
```

```
        log = open(DIR_PATH / "lab1.log", "a", buffering=1, encoding="utf-8")
        sys.stdout = log
        sys.stderr = log
        daemon_monitor()
        sys.exit(0)
    else:
        with open(PID_FILE, "w") as f:
            f.write(str(pid))
        print(f"[+] Режим защиты ВКЛЮЧЕН.")
        print(f"    - Существующих файлов защищено (chattr +i): {count}")
        print(f"    - {TBL_FILE.name} защищен от чтениямодификацииудаления//.")
        print(f"    - Демон мониторинга запущен (PID: {pid}).")


def disable_protection():
    check_root()

    pwd_hash, masks = load_templates()

    entered_pwd = getpass.getpass("Введите пароль для отключения защиты: ")
    if not verify_password(entered_pwd, pwd_hash):
        print("[!] Неверный пароль! Отказано в доступе.")
        sys.exit(1)

    if PID_FILE.exists():
        try:
            with open(PID_FILE, "r") as f:
                pid = int(f.read().strip())
            os.kill(pid, signal.SIGTERM)
        except ProcessLookupError:
            pass
        except Exception as e:
            print(f"[!] Ошибка остановки демона: {e}")
        finally:
            if PID_FILE.exists():
                PID_FILE.unlink()

    for item in DIR_PATH.iterdir():
        if item.is_file() and matches_any_mask(item.name, masks):
            set_immutable(item, False)

    set_immutable(TBL_FILE, False)

    print("[+] Режим защиты ВЫКЛЮЧЕН. Ограничения сняты.")


def show_status():
    is_active = PID_FILE.exists()
    status_str = "АКТИВЕН" if is_active else "ВЫКЛЮЧЕН"
```

```
    print(f"Статус защиты: {status_str}")
    if is_active:
        with open(PID_FILE, "r") as f:
            print(f"PID процесса мониторинга: {f.read().strip()}")


def main():
    parser = argparse.ArgumentParser(description="Система защиты файлов
    каталога по шаблонам")
    parser.add_argument("command", choices=["init", "enable", "disable", "status
    "],
                        help="Команда: init настройка(), enable включить(),
    disable выключить(), status статус()")
    args = parser.parse_args()

    if args.command == "init":
        init_program()
    elif args.command == "enable":
        enable_protection()
    elif args.command == "disable":
        disable_protection()
    elif args.command == "status":
        show_status()


if __name__ == "__main__":
    main()
```

Листинг 2. Конфигурация шаблонов и хеш пароля (template.tbl)

```
e12a2ac4a904fe760c4af98787702b99$e91fd47116551c81b5bc69ffd92ab89011c4a100df8c0
54bf553c5685df1fc03
*.secret
private.txt
```

Листинг 3. Клиентский веб-скрипт защиты контента (lab1.js)

```
const PASS_HASH = ”240
   be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9”;
const SPOOF_TEXT = ВНИМАНИЕ”: Копирование материалов с данного ресурса
   запрещено!”;

function isProtectionEnabled() {
    return localStorage.getItem(’site_protection_active’) !== ’false’;
}

async function sha256(text) {
    const encoder = new TextEncoder();
    const data = encoder.encode(text);
    const hashBuffer = await crypto.subtle.digest(’SHA-256’, data);
    return Array.from(new Uint8Array(hashBuffer))
        .map(b => b.toString(16).padStart(2, ’0’))
        .join(’’);
}

function preventDefaultAction(e) {
    if (isProtectionEnabled()) e.preventDefault();
}

function handleCopy(e) {
    if (!isProtectionEnabled()) return;
    e.preventDefault();
    if (e.clipboardData) {
        e.clipboardData.setData(’text/plain’, SPOOF_TEXT);
    }
}

function handleKeyDown(e) {
  if (!isProtectionEnabled()) return;
    // Ctrl+P / Cmd+P
    if ((e.ctrlKey || e.metaKey) && (e.key === ’p’ || e.key === ’P’)) {
        return;
    }

    e.preventDefault()
    console.log(e.key)
    return
```

```
}

window.addEventListener('blur', () => {
    if (isProtectionEnabled()) {
        document.body.classList.add('screen-protected');
    }
});
window.addEventListener('focus', () => {
    document.body.classList.remove('screen-protected');
});

document.addEventListener('contextmenu', preventDefaultAction);
document.addEventListener('selectstart', preventDefaultAction);
document.addEventListener('dragstart', preventDefaultAction);
document.addEventListener('copy', handleCopy);
document.addEventListener('cut', preventDefaultAction);
document.addEventListener('keydown', handleKeyDown);

const style = document.createElement('style');
style.innerHTML = `
    body.protected-mode {
        -webkit-user-select: none !important;
        -moz-user-select: none !important;
        -ms-user-select: none !important;
        user-select: none !important;
    }
    body.protected-mode img {
        pointer-events: none !important;
    }
    .screen-protected {
        filter: blur(25px) !important;
        transition: filter 0.1s ease;
    }

    .guard-panel {
        position: fixed;
        bottom: 15px;
        right: 15px;
        background: #222;
        color: #fff;
        padding: 10px 15px;
        border-radius: 8px;
        box-shadow: 0 4px 10px rgba(0,0,0,0.3);
        z-index: 999999;
        font-family: sans-serif;
        font-size: 13px;
        display: flex;
        align-items: center;
        gap: 10px;
```

```
    }
    .guard-btn {
        cursor: pointer;
        padding: 5px 10px;
        border: none;
        border-radius: 4px;
        font-weight: bold;
    }

    @media print {
        body, body.protected-mode, .screen-protected {
            user-select: auto !important;
            filter: none !important;
            visibility: visible !important;
        }
        .guard-panel, .no-print {
            display: none !important;
        }
    }
`;
document.head.appendChild(style);

function updateProtectionState() {
    const active = isProtectionEnabled();
    if (active) {
        document.body.classList.add('protected-mode');
    } else {
        document.body.classList.remove('protected-mode');
        document.body.classList.remove('screen-protected');
    }
    renderPanel();
}

async function toggleProtection() {
    const inputPass = promptВведите(" пароль для изменения режима защиты:");
    if (!inputPass) return;

    const hash = await sha256(inputPass);
    if (hash === PASS_HASH) {
        const newState = !isProtectionEnabled();
        localStorage.setItem('site_protection_active', newState ? 'true' : '
   false');
        updateProtectionState();
        alertРежим(` защиты ${newState ? ВКЛЮЧЕН'' : ОТКЛЮЧЕН''} для всех
   страниц!`);
    } else {
        alertНеверный(" пароль доступа!");
    }
}
```

```
function renderPanel() {
    let panel = document.getElementById('guard-panel');
    if (!panel) {
        panel = document.createElement('div');
        panel.id = 'guard-panel';
        panel.className = 'guard-panel no-print';
        document.body.appendChild(panel);
    }
    const active = isProtectionEnabled();
    panel.innerHTML = `
        <span>Защита: <b style="color:${active ? '#4ade80' : '#f87171'}">${
    active ? 'АКТИВНА' : 'ВЫКЛЮЧЕНА'}</b></span>
        <button class="guard-btn" style="background:${active ? '#ef4444' : '#22
    c55e'};color:#fff;" onclick="toggleProtection()">
            ${active ? 'Отключить' : 'Включить'}
        </button>
    `;
}

window.addEventListener('DOMContentLoaded', updateProtectionState);
```

Листинг 4. Разметка веб-страницы (index.html)

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Страница 1</title>
    <script src="lab1.js"></script>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; line-height: 1.6; }
        .card { border: 1px solid #ddd; padding: 20px; border-radius: 8px; max-
   width: 650px; }
    </style>
</head>
<body>
    <div class="card protected-content">
        <h1>Страница 1</h1>
        <p>Этот текст защищен от выделения, копирования в буфер и сохранения.</p
   >
        <svg width="200" height="80" style="background:#e2e8f0; border-radius:4
   px;">
            <circle cx="40" cy="40" r="25" fill="#3b82f6" />
            <text x="75" y="45" fill="#1e293b" font-weight="bold">Графика SVG</
   text>
        </svg>
        <p>Попробуйте скопировать этот текст.</p>
        <hr>
        <a href="page2.html">Перейти на Страницу 2 &rarr;</a>
    </div>
</body>
</html>
```

Листинг 5. Разметка веб-страницы (page2.html)

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Страница 2</title>
    <script src="lab1.js"></script>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; line-height: 1.6; }
        .card { border: 1px solid #ddd; padding: 20px; border-radius: 8px; max-
   width: 650px; }
    </style>
</head>
<body>
    <div class="card protected-content">
        <h1>Страница 2</h1>
        <p>Защита наследуется автоматически. Попробуйте нажать <code>Ctrl + S</
   code> для сохранения.</p>

        <svg width="200" height="80" style="background:#fef3c7; border-radius:4
   px;">
            <rect x="20" y="20" width="40" height="40" fill="#f59e0b" />
            <text x="75" y="45" fill="#78350f" font-weight="bold">Схема №2</text
   >
        </svg>

        <p>Для проверки печати нажмите <code>Ctrl + P</code> — страница
   корректно отправится на принтер.</p>
        <hr>
        <a href="index.html">&larr; Назад на Страницу 1</a> |
        <a href="page3.html">Перейти на Страницу 3 &rarr;</a>
    </div>
</body>
</html>
```

Листинг 6. Разметка веб-страницы (page3.html)

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Страница 3</title>
    <script src="lab1.js"></script>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; line-height: 1.6; }
        .card { border: 1px solid #ddd; padding: 20px; border-radius: 8px; max-
    width: 650px; }
    </style>
</head>
<body>
    <div class="card protected-content">
        <h1>Страница 3</h1>
        <p>Максимальная глубина ссылок достигнута. Данные защищены тем же
    скриптом.</p>

        <svg width="200" height="80" style="background:#dcfce7; border-radius:4
    px;">
            <polygon points="40,15 65,65 15,65" fill="#10b981" />
            <text x="75" y="45" fill="#064e3b" font-weight="bold">Графика №3</
    text>
        </svg>
        <hr>
        <a href="page2.html">&larr; Назад на Страницу 2</a>
    </div>
</body>
</html>
```